BOOTCAMP II

Planejamento Lógico TaskTracker

Entrega Inicial Fase 1

Estudante	Joaquim José Araújo Carvalho Filho
Curso	Análise e Desenvolvimento de Sistemas
Polo ou turma	CEUB Asa Norte
E-mail institucional	joaquim.jf@sempreceub.com.br

Brasília 2026

1 Apresentação e entendimento do problema

Este documento apresenta o planejamento lógico do TaskTracker, um sistema simples para registrar, organizar, acompanhar e concluir tarefas. O planejamento descreve o problema enfrentado pelo usuário, os dados necessários, as regras do sistema, o algoritmo em linguagem natural e a estrutura prevista para o repositório no GitHub. Essa definição servirá como base para a implementação em Python nas próximas fases do Bootcamp II.

1.1 Problema a ser resolvido

Uma pessoa ou equipe pode ter dificuldade para organizar as tarefas diárias quando utiliza anotações espalhadas, mensagens, papéis ou apenas a memória. Nesse cenário, prazos podem ser esquecidos, atividades importantes podem receber pouca atenção e não existe uma visão clara do que está pendente ou concluído. Em trabalhos coletivos, a falta de uma lista organizada também prejudica o acompanhamento das responsabilidades e da evolução das atividades.

O problema central é, portanto, a ausência de um local único e simples para registrar as tarefas e acompanhar sua situação. O usuário precisa visualizar rapidamente o que deve fazer, qual atividade é mais urgente, quando o prazo termina e o que já foi finalizado.

1.2 Solução proposta

O TaskTracker será um sistema de gerenciamento de tarefas executado inicialmente em modo texto. Ele permitirá cadastrar uma tarefa com título, descrição, prioridade e data limite; visualizar as tarefas registradas; marcar uma tarefa como concluída; consultar quantas estão pendentes ou concluídas; e encerrar o programa com segurança.

Ao centralizar essas informações, o sistema ajudará o usuário a definir prioridades, acompanhar prazos e evitar esquecimentos. A primeira versão será objetiva, mas sua estrutura permitirá futuras melhorias, como edição e exclusão de tarefas, filtros, autenticação de usuários, banco de dados e interface gráfica.

1.3 Objetivo do sistema

O objetivo do TaskTracker é oferecer uma forma prática e confiável de controlar tarefas, transformando informações fornecidas pelo usuário em uma lista organizada e fácil de acompanhar. O sistema deverá validar os dados antes de salvá-los e apresentar mensagens claras em cada operação.

2 Decomposição do problema e dados do sistema

O funcionamento do TaskTracker depende de dados de entrada, informações produzidas como saída e regras que garantem consistência. A decomposição abaixo separa esses elementos para facilitar a futura transformação do planejamento em código.

2.1 Entradas de dados

Dado	Descrição e exemplo
Título	Nome curto e obrigatório da tarefa. Exemplo: Entregar atividade do Bootcamp.
Descrição	Detalhamento opcional do que precisa ser realizado. Exemplo: Revisar o documento e enviar o PDF no AVA.
Prioridade	Nível de importância escolhido entre Alta, Média ou Baixa.
Data limite	Prazo para conclusão, informado no formato DD/MM/AAAA.
Identificador	Número gerado automaticamente pelo sistema para localizar cada tarefa.
Status	Situação controlada pelo sistema: Pendente no cadastro e Concluída após a finalização.
Opção do menu	Comando numérico usado para cadastrar, visualizar, concluir ou encerrar.

2.2 Saídas de dados

Saída	Informação exibida
Menu principal	Lista numerada das operações disponíveis no programa.
Confirmação	Mensagem informando que a tarefa foi cadastrada ou concluída com sucesso.
Lista de tarefas	Identificador, título, descrição, prioridade, data limite e status de cada tarefa.
Resumo	Quantidade total de tarefas, pendentes e concluídas.
Avisos de validação	Mensagens para título vazio, prioridade inválida, data incorreta ou opção inexistente.
Lista vazia	Mensagem explicando que ainda não existem tarefas cadastradas.

2.3 Regras do sistema

- O título é obrigatório e não pode conter apenas espaços.
- A prioridade deve ser somente Alta, Média ou Baixa.
- A data limite deve ser informada no formato DD/MM/AAAA e representar uma data válida.
- Toda nova tarefa deve receber um identificador único e o status inicial Pendente.
- Somente uma tarefa existente e ainda pendente pode ser marcada como concluída.
- O status deve assumir apenas os valores Pendente ou Concluída.
- Quando não existirem tarefas, o sistema deve informar isso em vez de mostrar uma lista vazia sem explicação.
- Uma opção inválida no menu não deve encerrar o programa; o sistema deve orientar o usuário e exibir o menu novamente.

3 Receita lógica do sistema

O algoritmo a seguir descreve o comportamento esperado do TaskTracker em uma sequência compreensível. O programa mantém uma lista de tarefas em memória enquanto estiver em execução e repete o menu principal até que o usuário escolha sair.

3.1 Inicialização e menu principal

1. Iniciar o programa.
2. Criar uma lista vazia chamada lista de tarefas.
3. Definir o próximo identificador com o valor 1.
4. Exibir o nome TaskTracker e as opções: 1 Cadastrar tarefa, 2 Visualizar tarefas, 3 Concluir tarefa e 4 Sair.
5. Solicitar ao usuário uma opção.
6. Executar a função correspondente à opção escolhida.
7. Se a opção não existir, mostrar uma mensagem de erro.
8. Depois de concluir a operação, voltar ao menu principal.
9. Repetir esse processo até que o usuário escolha a opção 4.

3.2 Cadastro de uma tarefa

10. Pedir o título da tarefa.
11. Remover espaços extras do início e do fim do título.
12. Verificar se o título ficou vazio. Se estiver vazio, informar o erro e solicitar novamente.
13. Pedir a descrição da tarefa. A descrição pode ficar vazia.
14. Pedir a prioridade.
15. Verificar se a prioridade é Alta, Média ou Baixa. Se for diferente, informar o erro e solicitar novamente.
16. Pedir a data limite no formato DD/MM/AAAA.
17. Verificar se a data informada existe e está no formato correto. Se estiver incorreta, solicitar novamente.
18. Criar a tarefa com o próximo identificador disponível, os dados informados e o status Pendente.
19. Adicionar a tarefa à lista de tarefas.
20. Aumentar o próximo identificador em uma unidade.
21. Exibir a mensagem Tarefa cadastrada com sucesso.

3.3 Visualização das tarefas

22. Verificar se a lista de tarefas está vazia.
23. Se estiver vazia, mostrar a mensagem Nenhuma tarefa cadastrada e retornar ao menu.
24. Se houver tarefas, percorrer a lista do primeiro ao último item.
25. Para cada tarefa, exibir identificador, título, descrição, prioridade, data limite e status.
26. Contar quantas tarefas estão pendentes e quantas estão concluídas.
27. Ao final, exibir o total de tarefas e os totais por status.
28. Retornar ao menu principal.

3.4 Conclusão de uma tarefa

29. Verificar se existe alguma tarefa cadastrada.
30. Se não existir, informar o usuário e retornar ao menu.
31. Mostrar a lista de tarefas e pedir o identificador da tarefa que será concluída.
32. Procurar na lista uma tarefa com o identificador informado.
33. Se a tarefa não for encontrada, mostrar uma mensagem de erro.
34. Se a tarefa já estiver concluída, informar que nenhuma alteração é necessária.
35. Se a tarefa estiver pendente, alterar seu status para Concluída.
36. Exibir a mensagem Tarefa concluída com sucesso e retornar ao menu.

3.5 Encerramento

37. Quando o usuário escolher a opção 4, exibir uma mensagem de encerramento.
38. Finalizar a repetição do menu.
39. Encerrar o programa.

3.6 Pseudocódigo geral

```
INÍCIO
    tarefas <- lista vazia
    proximo_id <- 1

    REPETIR
        mostrar menu
        ler opcao

        SE opcao = 1 ENTÃO
            validar título, prioridade e data
            criar tarefa com status Pendente
            adicionar tarefa à lista
            proximo_id <- proximo_id + 1
        SENÃO SE opcao = 2 ENTÃO
            mostrar tarefas e resumo por status
        SENÃO SE opcao = 3 ENTÃO
            localizar tarefa pelo identificador
            alterar status para Concluída
        SENÃO SE opcao = 4 ENTÃO
            mostrar mensagem de encerramento
        SENÃO
            mostrar mensagem de opção inválida
        FIM SE
    ATÉ opcao = 4
FIM
```

4 Mapeamento arquitetural para Git e GitHub

Na Fase 2, o projeto será colocado em um repositório Git. A separação entre documentação, código-fonte e testes ajudará a manter os arquivos compreensíveis e facilitará o acompanhamento das alterações.

4.1 Árvore planejada do repositório

```
tasktracker-projeto/
|-- README.md
|-- .gitignore
|-- docs/
|   `-- planejamento_logico.pdf
|-- src/
|   `-- main.py
`-- tests/
    `-- test_tasktracker.py
```

4.2 Finalidade dos arquivos e pastas

Item	Finalidade
README.md	Apresentar o projeto, seu objetivo, funcionalidades e instruções para execução.
.gitignore	Impedir o envio de arquivos temporários, cache e configurações locais ao repositório.
docs	Armazenar o planejamento lógico e outras documentações do sistema.
src	Guardar o código-fonte do TaskTracker, começando pelo arquivo main.py.
tests	Reunir testes que verifiquem regras e funcionalidades do programa.

4.3 Plano de aprendizado e versionamento

- Instalar e configurar o Git no computador.
- Criar uma conta no GitHub ou acessar uma conta existente.
- Criar o repositório tasktracker-projeto e preparar as pastas planejadas.
- Executar git init para iniciar o controle de versão local.
- Usar git status para acompanhar arquivos novos ou modificados.

- Selecionar alterações com git add e registrá-las com mensagens claras em git commit.
- Conectar o repositório local ao GitHub e publicar as alterações com git push.
- Criar branches para desenvolver novas funcionalidades sem alterar imediatamente a versão principal.
- Aprender a comparar versões, corrigir conflitos simples e consultar o histórico de commits.

4.4 Organização prevista dos commits

Etapa	Exemplo de mensagem de commit
Estrutura inicial	chore: cria estrutura inicial do projeto
Menu	feat: adiciona menu principal
Cadastro	feat: implementa cadastro e validação de tarefas
Visualização	feat: exibe lista e resumo de tarefas
Conclusão	feat: permite concluir tarefa pelo identificador
Testes	test: adiciona testes das regras do TaskTracker
Documentação	docs: atualiza instruções de uso no README

5 Link da apresentação em vídeo

A apresentação deverá mostrar este planejamento, explicar o problema resolvido pelo TaskTracker, descrever o algoritmo e apresentar a estrutura planejada para o GitHub.

Link do vídeo no YouTube

_____

Requisitos para a gravação

- Duração aproximada de 5 minutos.
- Gravação horizontal no formato 16 por 9, sem utilizar YouTube Shorts.
- Rosto do estudante visível durante a apresentação.
- Áudio claro e inteligível.

- Vídeo publicado como Público ou Não listado.
- Link ativo inserido neste documento antes da exportação final para PDF.